Programowanie i metody numeryczne

Ćwiczenia 3.

Algorytmy. Pliki

In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
```

Algorytmy

W tej części zajmiemy się analizą złożoności obliczeniowej algorytmów.

Zadanie 1. Sortowanie.

Sortowanie to zadanie polegające na uporządkowaniu elementów pewnego zbioru, na którym określona jest relacja porządku. Sortować możemy więc na przykład liczby rzeczywiste, posługując się naturalnym porządkiem $\leq$, oraz słowa, wykorzystując porządek alfabetyczny.

Istnieje wiele algorytmów sortowania. Przyjrzymy się trzem z nich:

- sortowanie przez wybieranie (ang. *selection sort*),
- sortowanie szybkie (ang. *quick sort*),
- sortowanie przez scalananie (ang. *merge sort*).

1. Wykorzystując sztuczną inteligencję lub materiały dostępne w internecie zapoznaj się z wymienionymi algorytmami sortowania. Oblicz czasową i pamięciową złożoność obliczeniową każdego z tych algorytmów w najlepszym, średnim i najgorszym przypadku.

2. Poniżej znajdziesz przykładowe implementacje każdego z tych algorytmów. Funkcje te przyjmują jako argument jednowymiarową tablicę NumPy i zwracają posortowaną tablicę o tych samych elementach. Przyjrzyj się im i postaraj się je zrozumieć.

In [ ]:
```
def selection_sort(arr: np.ndarray) -> None:
    """
    Sortuje tablicę NumPy in-place za pomocą algorytmu selection sort.
    """
    n = len(arr)
    for i in range(n):
        # Zakładamy, że minimalny element jest na pozycji i
        min_idx = i

        # Szukamy najmniejszego elementu w pozostałej części tablicy
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j

        # Zamieniamy elementy miejscami
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

In [ ]:
```
# Przykład użycia:
a = np.array([64, 25, 12, 22, 11])
selection_sort(a)
print(a)  # wynik: [11 12 22 25 64]
```

In [ ]:
```
def merge_sort(arr: np.ndarray) -> None:
    """
    Sortuje tablicę NumPy in-place za pomocą algorytmu merge sort.
    """
    def _merge_sort(a, left, right):
        if left < right:
            mid = (left + right) // 2
            _merge_sort(a, left, mid)
```

```
            _merge_sort(a, mid + 1, right)
            _merge(a, left, mid, right)

    def _merge(a, left, mid, right):
        # Tworzymy kopie podtablic
        left_part = a[left:mid+1].copy()
        right_part = a[mid+1:right+1].copy()

        i = j = 0
        k = left

        # Scalanie dwóch posortowanych części
        while i < len(left_part) and j < len(right_part):
            if left_part[i] <= right_part[j]:
                a[k] = left_part[i]
                i += 1
            else:
                a[k] = right_part[j]
                j += 1
            k += 1

        # Jeśli zostały elementy w lewej części
        while i < len(left_part):
            a[k] = left_part[i]
            i += 1
            k += 1

        # Jeśli zostały elementy w prawej części
        while j < len(right_part):
            a[k] = right_part[j]
            j += 1
            k += 1

    _merge_sort(arr, 0, len(arr) - 1)
```

In [ ]:

```
# Przykład użycia:
a = np.array([38, 27, 43, 3, 9, 82, 10])
merge_sort(a)
print(a)  # wynik: [ 3  9 10 27 38 43 82 ]
```

In [ ]:
```python
def quick_sort(arr: np.ndarray) -> None:
    """
    Sortuje tablicę NumPy in-place za pomocą algorytmu quick sort.
    """
    def _quick_sort(a, low, high):
        if low < high:
            pivot_index = _partition(a, low, high)
            _quick_sort(a, low, pivot_index - 1)
            _quick_sort(a, pivot_index + 1, high)

    def _partition(a, low, high):
        pivot = a[high]  # wybieramy ostatni element jako pivot
        i = low - 1

        for j in range(low, high):
            if a[j] <= pivot:
                i += 1
                a[i], a[j] = a[j], a[i]

        # umieszczamy pivot na właściwym miejscu
        a[i + 1], a[high] = a[high], a[i + 1]
        return i + 1

    _quick_sort(arr, 0, len(arr) - 1)
```

In [ ]:
```python
# Przykład użycia:
a = np.array([10, 7, 8, 9, 1, 5])
quick_sort(a)
print(a)  # wynik: [1 5 7 8 9 10]
```

3. Wykorzystując te funkcje, napisz kod wczytujący ze standardowego wejścia liczbę naturalną N. Twój kod powinien następnie N--krotnie, dla $n = 1, 2, \ldots, N$, tworzyć n--elementową, jednowymiarową tablice losowych liczb rzeczywistych, a następnie sortować tę tablicę czterema sposobami: każdą z powyższych funkcji oraz za pomocą odpowiedniej funkcji bibliotecznej, mierząc czas wykonywania każdej operacji sortowania. Na koniec kod powienien wykonać rysunek przedstawiający wykresy czasu wykonywania naszych funkcji sortujących oraz funkcji bibliotecznej w zależności od n.

In [ ]:
```
# Miejsce na Twoje rozwiązanie
```

4. Przeanalizuj wyniki dla różnych wartości N. Która z funkcji jest najszybsza? Czy zależność czasu sortowania od rozmiaru sortowanej tablicy odzwierciedla złożoności obliczeniowe poszczególnych algorytmów?

Pliki

W tej części zajmiemy się odczytywaniem danych z plików i zapisywaniem ich do plików.

Zadanie 2. Plik tekstowy.

Napisz kod, który wczyta ze standardowego wejścia listę dowolnie wielu oddzielonych spacjami imion, a następnie zapisze te imiona w pliku `imiona.txt`, każde w nowej linii.

In [ ]:
```
# Miejsce na Twoje rozwiązanie
```

Napisz kod, którego zadaniem będzie analiza utworzonego w ten sposób pliku `imiona.txt`. Twój kod powinien wypisać na standardowe wyjście liczbę imion zapisanych w tym pliku, a następnie wszystkie imiona, oddzielone spacjami.

In [ ]:
```
# Miejsce na Twoje rozwiązanie
```

Zajmiemy się teraz filtrowaniem i sortowaniem danych zapisanych w pliku. Napisz kod, który przeanalizuje plik `imiona.txt` i na jego podstawie utworzy nowe pliki:

- `imiona_BG.txt`, zawierający te imiona z pliku `imiona.txt`, które zaczynają się na literę *B* lub *G*, ułożone w takiej kolejności, w jakiej występują w pliku `imiona.txt`, każde w nowej linii,
- `imiona_a.txt`, zawierający te imiona z pliku `imiona.txt`, które kończą się na literę *a*, posortowane alfabetycznie, każde w nowej linii,

- `imiona_dlugie.txt`, zawierający te imiona z pliku `imiona.txt`, które mają co najmniej pięć liter, posortowane rosnąco według długości, każde w nowej linii.

Jeśli okaże się, że żadne z imion w pliku `imiona.txt` nie spełnia kryteriów wymienionych w którymś z powyższych punktów, Twój kod nie powinien tworzyć pliku wymienionego w tym punkcie. Zamiast tego powinien wypisać na standardowe wyjście stosowny komunikat.

In [ ]:
```
# Miejsce na Twoje rozwiązanie
```

Zadanie 3. Statystyka tekstu.

Poniższy kod tworzy plik `nowela.txt` zawierający losową liczbę automatycznie wygenerowanych zdań w języku polskim (niekoniecznie sensownych).

In [ ]:
```
import random

# Elementy zdań
subjects = [
    "Kot", "Pies", "Programista", "Nauczyciel", "Student",
    "Moja siostra", "Mój sąsiad", "Samochód", "Komputer", "Dziecko"
]

verbs = [
    "lubi", "czyta", "pisze", "ogląda", "tworzy",
    "kupuje", "sprząta", "odkrywa", "buduje", "analizuje"
]

objects = [
    "książki", "programy komputerowe", "ciekawe filmy", "nowe technologie",
    "smaczne obiady", "stare fotografie", "trudne zadania",
    "piękne krajobrazy", "różne projekty", "muzykę klasyczną"
]

# Losowanie liczby zdań do wygenerowania - między 3 a 10
num_sentences = random.randint(3, 10)
```

```python
# Generowanie zdań
sentences = []
for _ in range(num_sentences):
    subject = random.choice(subjects)
    verb = random.choice(verbs)
    obj = random.choice(objects)

    sentence = f"{subject} {verb} {obj}."
    sentences.append(sentence)

# Zapis do pliku
with open("nowela.txt", "w", encoding="utf-8") as file:
    for sentence in sentences:
        file.write(sentence + " ")

print("Plik nowela.txt został utworzony.")
```

Napisz kod, który przeanalizuje plik `nowela.txt` i wypisze na standardowe wyjście liczbę zawartych w nim:

- zdań (*wskazówka: plik zawiera wyłącznie zdania oznajmujące, zatem liczba zdań jest równa liczbie kropek*),
- słów (*wskazówka: słowo można zdefiniować jako fragment tekstu ograniczony znakami spacji*),
- liter, czyli znaków od `A` do `Z` i od `a` do `z` (*wskazówka: wykorzystaj metodę* `isalpha`),
- wszystkich znaków.

In [ ]:
```python
# Miejsce na Twoje rozwiązanie
```

Zadanie 4. Plik CSV.

CSV (ang. *Comma-separated values - wartości oddzielone przecinkami*) to tekstowy format przechowywania danych tabelarycznych. Kolejne wiersze pliku w formacie CSV odpowiadają kolejnym wierszom tabeli (jednak pierwszy wiersz pliku CSV zawiera zwykłe nagłówki kolumn), zaś w każdym wierszu dane z kolejnych kolumn oddzielone są przecinkami (lub innymi, z góry określonymi separatorami, np. średnikami lub tabulatorami).

Na przykład, tabela:

Imię i nazwisko	Miasto	E-mail
Anna Kowalska	Warszawa	anna.kowalska@example.com
Piotr Nowak	Kraków	piotr.nowak@example.com
Magda Wójcik	Gdańsk	magda.wojcik@example.com
Tomasz Zieliński	Poznań	tomasz.zielinski@example.com

w formacie CSV może przyjąć następującą postać:

```
CSV
imie_nazwisko,miasto,email
Anna Kowalska,Warszawa,anna.kowalska@example.com
Piotr Nowak,Kraków,piotr.nowak@example.com
Magda Wójcik,Gdańsk,magda.wojcik@example.com
Tomasz Zieliński,Poznań,tomasz.zielinski@example.com
```

Choć nie jest to ściśle konieczne, zaleca się, by nazwy kolumn w formacie CSV zawierały wyłącznie małe litery alfabetu łacińskiego i podkreślniki, w przeciwnym razie część oprogramowania do analizy plików CSV może nie działać poprawnie. Właśnie dlatego kolumna *Imię i nazwisko* została zapisana w formacie CSV jako `imie_nazwisko` etc.

Ze względu na swoją prostotę i uniwersalność, format CSV jest szeroko stosowany w analizie danych. Pozwala na łatwą wymianę informacji między różnymi narzędziami, w tym arkuszami kalkulacyjnymi i bazami danych.

Praca z plikami w formacie CSV w języku Python jest możliwa dzięki modułowi `csv`.

In [ ]:
```
import csv
```

Alternatywnie można posłużyć się jedną z zaawansowanych bibliotek do analizy danych, np. Pandas lub Polars. Nie będziemy się tu jednak nimi zajmować.

Poniższy kod tworzy słownik `students`, przechowujący wyniki pewnego testu. Kluczami są w nim imiona studentów, zaś wartościami - liczby uzyskanych przez nich punktów. Długość słownika, klucze i wartości są wybierane losowo.

In [ ]:
```python
import random

names = [
    "Adrian", "Agata", "Agnieszka", "Alicja", "Amelia", "Andrzej", "Anna", "Antoni",
    "Barbara", "Bartłomiej", "Beata", "Benedykt", "Bogdan", "Bogumiła", "Bronisław",
    "Czesław", "Damian", "Dobrosława", "Dorota", "Emilia", "Ewelina", "Fabian",
    "Filip", "Franciszek", "Gabriela", "Grażyna", "Grzegorz", "Hanna", "Hubert",
    "Igor", "Izabela", "Jan", "Jakub", "Joanna", "Józef", "Kacper", "Karol",
    "Karolina", "Katarzyna", "Klaudia", "Krzysztof", "Leokadia", "Leon", "Lena",
    "Ludwik", "Łukasz", "Magdalena", "Maja", "Maria", "Marek", "Marcin",
    "Mateusz", "Michał", "Mirosława", "Miłosz", "Monika", "Natalia",
    "Oliwia", "Oskar", "Patrycja", "Patryk", "Paweł", "Piotr", "Rafał",
    "Radosław", "Sebastian", "Stanisław", "Sylwia", "Szymon", "Tadeusz",
    "Teofil", "Tomasz", "Weronika", "Wiktoria", "Władysław", "Wojciech", "Zofia"
]

dict_size = random.randint(1, 10)

students = {}
for name in random.sample(names, dict_size):
    students[name] = random.randint(0, 10)

print(students)
```

Napisz kod, który zapisze dane ze słownika `students` w pliku `students.csv`. Plik powinien zawierać dwie kolumny, `student` i `points`. Kolejne wiersze pliku powinny odpowiadać kolejnym elementom słownika.

In [ ]:
```python
# Miejsce na Twoje rozwiązanie
```

Napisz kod, który przeanalizuje plik `students.csv` i wypisze na standardowe wyjście średnią ocen oraz listę studentów, którzy uzyskali najwyższy wynik (pamiętaj, że najwyższy wynik to niekoniecznie 10 punktów).

In [ ]:
```python
# Miejsce na Twoje rozwiązanie
```

Zadanie 5. Dane w nazwach plików.

Niejednokrotnie sama nazwa pliku również jest źródłem danych. Na przykład, treść pliku może zawierać wyniki obliczeń lub pomiarów, zaś jego nazwa - wartości pewnych parametrów, przy których powstała seria wyników zapisana w pliku.

Folder `tmp_Warszawa` zawiera wyniki pomiarów temperatury powietrza w prowadzonej przed Instytut Meteorologii i Gospodarki Wodnej – Państwowy Instytut Badawczy stacji synoptycznej Warszawa-Okęcie w latach 2000 - 2025. Wyniki te są zapisane w wielu plikach CSV - każdy z nich przechowuje dane dla konkretnego roku i konkretnego miesiąca. Nazwy plików mają postać

`tmp_rok_miesiac.csv`

gdzie `rok` to czteroznakowy łańcuch tekstowy zawierający numer roku (np. `2025`), zaś `miesiac` - dwuznakowy łańcuch tekstowy podający numer miesiąca (np. `03` dla marca). Pliki pogrupowane są w foldery odpowiadające poszczególnym latom, np. plik `tmp_2010_10.csv` znajduje się w podfolderze `tmp_2010`.

Pliki zawierają informacje o: kodzie stacji meteorologicznej i jej nazwie (które są zawsze takie same - a więc nie będą istotne), dacie pomiaru oraz maksymalnej, minimalnej i średniej dobowej temperaturze powietrza. Zajrzyj do wybranych z nich, by zapoznać się z układem kolumn (ich nazwy powinny być czytelne).

Napisz funkcję `parse_filename`, przyjmującą jako argument nazwę pliku i zwracającą rok i miesiąc, dla których wykonano pomiary.

In [ ]:
```
def parse_filename(filename: str) -> tuple[int, int]:
    # Miejsce na Twoje rozwiązanie

    return year, month
```

Napisz funkcję `get_tmp`, przyjmującą jako argument numery roku, miesiąca i dnia oraz zwracającą wartości maksymalnej, minimalnej i średniej temperatury dobowej we wskazanym dniu.

In [ ]:
```
def get_tmp(year: int, month: int, day: int) -> tuple[float, float, float]:
    # Miejsce na Twoje rozwiązanie

    return temp_max, temp_min, temp_avg
```

Napisz kod, który wczytuje ze standardowego wejścia numer roku i miesiąca, a następnie rysuje wykres maksymalnej (kolor czerwony), minimalnej (kolor niebieski) i średniej (kolor żółty) dobowej temperatury w danym miesiącu.

In [ ]:
```
# Miejsce na Twoje rozwiązanie
```

Napisz kod, który wczytuje ze standardowego wejścia numer miesiąca, a następnie rysuje wykres maksymalnej (kolor czerwony), minimalnej (kolor niebieski) i średniej (kolor żółty) dobowej temperatury w danym miesiącu, uśredniając ich wartości po wszystkich dostępnych latach.

In [ ]:
```
# Miejsce na Twoje rozwiązanie
```

Praca domowa

Tu już niebawem znajdziesz pracę domową.